NutriQuery

Food & Nutrition Intelligence System

Complete System Documentation

ISE305 — Database Systems Project

May 21, 2026

A comprehensive reference covering the complete architecture, database design, backend API, machine learning pipeline, frontend interface, and testing infrastructure.

Contents

1 Project Overview **5**
1.1 Technology Stack . . . 5
1.2 System Capabilities . . . 5

2 System Architecture & Program Flow **5**

3 Database Design **6**
3.1 Entity-Relationship Model . . . 6
3.2 Normalization Process . . . 6
3.3 3NF Compliance Verification . . . 7
3.4 Complete Table Definitions . . . 8
3.4.1 Table: Brands . . . 8
3.4.2 Table: FOOD_CATEGORY . . . 8
3.4.3 Table: DATA_TYPE . . . 8
3.4.4 Table: Foods . . . 9
3.4.5 Table: Nutrition_Metrics . . . 11
3.4.6 Table: HEALTH_SCORE . . . 12
3.4.7 Table: ALLERGEN_PROFILE . . . 13
3.4.8 Table: ML_Predictions . . . 15
3.5 Data Value Creation . . . 16

4 Backend Documentation **16**
4.1 File: `database.py` — Connection Management . . . 16
4.1.1 DB_CONFIG (Module Constant) . . . 17
4.1.2 get_connection() . . . 17
4.1.3 get_db() . . . 17
4.2 File: `schemas.py` — Pydantic Data Models . . . 17
4.2.1 Brand Schemas . . . 17
4.2.2 Nutrition Schemas . . . 17
4.2.3 Health Score Schemas . . . 18
4.2.4 Allergen Profile Schemas . . . 18
4.2.5 ML Prediction Schemas . . . 18
4.2.6 Food Schemas . . . 18
4.2.7 AggregationResult Schema . . . 18
4.3 File: `crud.py` — Data Access Layer . . . 18
4.3.1 Shared Query Infrastructure . . . 19
4.3.2 _build_food_dict(row) . . . 19
4.3.3 get_food(conn, cursor, fdc_id) . . . 19
4.3.4 _update_row(conn, cursor, table, columns, fdc_id, data) . . . 19
4.3.5 update_nutrition(conn, cursor, fdc_id, data) . . . 20
4.3.6 update_health_score(conn, cursor, fdc_id, data) . . . 20
4.3.7 update_allergen(conn, cursor, fdc_id, data) . . . 20
4.3.8 get_foods_by_range(conn, cursor, min_health_score, max_sodium, max_carbs, limit=100) . . . 20
4.3.9 get_foods_by_diet(conn, cursor, no_gluten, no_dairy, limit=100) 20
4.3.10 get_category_aggregation(conn, cursor, category) . . . 20

4.3.11 get_foods_with_missing_data(conn, cursor, limit=100) 20
4.3.12 create_brand(conn, cursor, data) 20
4.3.13 get_brands(conn, cursor, skip, limit) 21
4.3.14 search_foods(conn, cursor, name, limit) 21
4.3.15 get_foods_browse(conn, cursor, skip, limit, name) 21
4.3.16 get_all_foods(conn, cursor, skip, limit) 21
4.3.17 get_categories(conn, cursor) 21
4.3.18 get_predictions(conn, cursor, limit) 21
4.4 File: `main.py` — API Endpoints 21
4.4.1 Application Setup 21
4.4.2 Endpoint Catalog 22
4.4.3 Import State Tracking 23
4.5 File: `ml_service.py` — Machine Learning Pipeline 23
4.5.1 Device Selection: _get_device() 23
4.5.2 Model Architecture: NutriScorePredictor 23
4.5.3 _compute_normalization_stats(cursor) 23
4.5.4 _train_model(cursor) 24
4.5.5 run_inference_and_store(conn, cursor) 24
4.5.6 delete_predictions(conn, cursor) 24
4.5.7 ML Router 24
4.6 File: `data_import.py` — CSV Data Import 24
4.6.1 Value Helpers 24
4.6.2 _resolve_or_create(cursor, cache, table, id_col, name_col, name_value, extra_cols=None) 25
4.6.3 _load_usda_csv(conn, cursor, stats) 25
4.6.4 _load_health_csv(conn, cursor) 25
4.6.5 import_all_data() 25

5 Frontend Documentation **26**
5.1 File: `api.js` — API Client 26
5.1.1 fetchAPI(endpoint, options) 26
5.2 File: `App.jsx` — Application Shell 26
5.3 File: `BrowseSection.jsx` — Food Browser 26
5.3.1 loadPage(pageNum, searchTerm) 26
5.3.2 onRowClicked(event) 27
5.3.3 openEdit(section) / saveEdit() 27
5.3.4 AG Grid Configuration 27
5.4 File: `AnalyticsSection.jsx` — Nutrition Analytics 27
5.4.1 Range Query (Req 4) 27
5.4.2 Dietary Filtering (Req 5) 27
5.4.3 Category Aggregation (Req 6) 28
5.4.4 Gap Identification (Req 7) 28
5.5 File: `MLEngineSection.jsx` — ML Engine 28
5.6 File: `BrandsSection.jsx` — Brand Management 28
5.7 File: `DataImportSection.jsx` — Data Import 28
5.8 File: `index.css` — Design System 28

6 Testing Infrastructure **29**
6.1 File: `conftest.py` — Test Fixtures 29
6.1.1 test_db_setup (Session Scope) 29
6.1.2 db_conn / db_cursor (Function Scope) 29
6.1.3 client (Function Scope) 29
6.1.4 seed_test_data 29
6.2 Test Files 29

7 API Reference Summary **30**

8 Data Flow Diagrams **30**
8.1 Import Flow 30
8.2 Query Flow 31
8.3 ML Flow 31
8.4 Edit Flow 31

9 Index of Functions **32**

1 Project Overview

NutriQuery is a full-stack food and nutrition intelligence platform built on a Microsoft SQL Server (MSSQL) database with a FastAPI Python backend, a React + Vite frontend, and a PyTorch-based machine learning pipeline. The system ingests USDA FoodData Central and OpenFoodFacts datasets, normalizes them into a Third Normal Form (3NF) schema, exposes a RESTful API for querying and correcting food data, and provides ML-predicted Nutri-Score classifications.

1.1 Technology Stack

- **Database:** Microsoft SQL Server 2022 (Docker container)
- **Backend:** Python 3.12, FastAPI, pymssql, PyTorch, scikit-learn
- **Frontend:** React 18, Vite 5, AG Grid Community
- **ML:** PyTorch feed-forward neural network (5→32→16→5)
- **Testing:** pytest (48 tests, real database integration)
- **Data Sources:** USDA comprehensive_foods_usda.csv, OpenFoodFacts foods_health_scores_al

1.2 System Capabilities

1. **Bulk Data Import** — Ingest ∼40,000 food records from CSV into normalized 3NF schema
2. **Record Retrieval** — Fetch complete food profiles with all related data via 7-table JOIN
3. **Data Correction** — Update nutrition metrics, health scores, and allergen profiles
4. **Range Querying** — Filter foods by health score, sodium, and carbohydrate thresholds
5. **Dietary Filtering** — Find gluten-free and dairy-free foods via allergen profiles
6. **Category Aggregation** — Compute average nutritional statistics per food category
7. **Gap Identification** — Surface records with missing or incomplete nutrition data
8. **Metadata Management** — CRUD operations for food brands
9. **ML Inference** — Train a neural network on labeled data; predict Nutri-Score for all foods
10. **Browse & Search** — Paginated food browser with nutrition-at-a-glance; inline editing

2 System Architecture & Program Flow

The system follows a layered architecture with clear separation between data, business logic, API, and presentation layers. The program flow, ordered by execution dependency, is:

1. **Database Initialization** — `db/init.sql` creates all 8 tables with constraints and indexes
2. **Data Import** — `data_import.py` reads CSV files and populates the normalized schema

3. **Connection Management** — `database.py` provides connection pooling and FastAPI dependency injection
4. **Data Schemas** — `schemas.py` defines Pydantic models for request/response validation
5. **CRUD Operations** — `crud.py` contains all SQL query logic, exposed as pure functions
6. **API Endpoints** — `main.py` maps HTTP routes to CRUD functions; includes ML router
7. **ML Pipeline** — `ml_service.py` trains and runs inference using PyTorch
8. **Frontend** — React components consume the API via `api.js`; render with AG Grid

Layer	Technology	Files
Presentation	React + Vite + AG Grid	`frontend/src/*.jsx`
API Gateway	FastAPI + CORS	`main.py`
Business Logic	Python + PyTorch	`crud.py, ml_service.py, data_import.py`
Data Access	pymssql	`database.py`
Data Store	MSSQL (Docker)	`db/init.sql`
Validation	Pydantic	`schemas.py`
Testing	pytest (real DB)	`tests/*.py`

Figure 1: System Architecture Layers

3 Database Design

3.1 Entity-Relationship Model

The database follows strict Third Normal Form (3NF) with 8 tables. The design uses Chen's notation with the following principles:

- Every table has a single-column primary key (natural or surrogate IDENTITY)
- All non-key attributes are fully functionally dependent on the primary key (2NF)
- No transitive dependencies exist — lookup values are extracted to separate tables (3NF)
- One-to-one relationships use UNIQUE foreign keys
- One-to-many relationships use standard foreign keys
- All foreign keys have referential integrity constraints with CASCADE or SET NULL actions

3.2 Normalization Process

The schema was normalized from an earlier design through the following transformations:

1. **Extracted FOOD_CATEGORY entity.** The original `Foods` table stored `food_category` as a VARCHAR string (e.g., "Snacks", "Dairy and Egg Products"). This created update anomalies — changing a category name required updating every row. Solution: create a `FOOD_CATEGORY` lookup table with `category_id` (IDENTITY PK) and `category_name` (UNIQUE). The `Foods` table references it via `category_id` FK.

Relationship	Cardinality
Brand → Food	1:N (optional — unbranded foods have NULL brand_id)
Food_Category → Food	1:N (required)
Data_Type → Food	1:N (required)
Food → Nutrition_Metrics	1:1 (UNIQUE fdc_id)
Food → Health_Score	1:1 (UNIQUE fdc_id)
Food → Allergen_Profile	1:1 (UNIQUE fdc_id)
Food → ML_Predictions	1:N

Figure 2: Entity Relationships

2. **Extracted DATA_TYPE entity.** Same pattern as categories. The `data_type` VARCHAR column ("SR Legacy", "Survey (FNDDS)") was extracted to a `DATA_TYPE` lookup table with `type_id` and `type_name`.

3. **Removed ecoscore_grade from Brands.** Eco-score depends on individual food products, not the brand as a whole. Different products from the same brand can have different eco-scores. This was a transitive dependency (`brand_id` does not functionally determine `ecoscore_grade`), violating 3NF. The column was dropped.

4. **Decomposed Health_and_Allergens into two entities.** The original combined table mixed scoring metrics (`health_score`, `nutriscore_grade`, `nova_group`) with intrinsic food properties (`contains_gluten`, `contains_dairy`). These represent independent functional domains. Solution: split into `HEALTH_SCORE` (scoring metrics derived from nutrition) and `ALLERGEN_PROFILE` (binary food properties independent of scoring).

3.3 3NF Compliance Verification

- **1NF:** All attributes store atomic values. No multi-valued or composite attributes exist.
- **2NF:** Every table has a single-column primary key. No partial dependencies can exist.
- **3NF:** Transitive dependencies eliminated: `food_category` and `data_type` extracted to lookup tables; `ecoscore_grade` removed from Brands; Health/Allergen split into separate entities.

3.4 Complete Table Definitions

3.4.1 Table: Brands

Column	Type	Constraints	Default	Description
brand_id	INT	PK, IDENTITY(1,1)	auto	Auto-generated unique brand identifier
brand_name	VARCHAR(255)	NOT NULL	—	Display name of the brand
brand_owner	VARCHAR(255)	NULL	—	Parent company or brand owner

Purpose: Stores food brand metadata. One brand can produce many foods (1:N relationship). Foods without a brand (generic/unbranded items) store NULL in their brand_id FK.

Indexes: None beyond PK (small table, typically $\ll$ 1,000 rows).

3.4.2 Table: FOOD_CATEGORY

Column	Type	Constraints	Default	Description
category_id	INT	PK, IDENTITY(1,1)	auto	Auto-generated category identifier
category_name	VARCHAR(255)	NOT NULL, UNIQUE	—	Food category name (e.g., Snacks, Dairy)

Purpose: Normalized lookup table for food categories. Eliminates VARCHAR duplication in the Foods table. A food's category is referenced by category_id FK.

3NF Rationale: Extracted from `Foods.food_category` VARCHAR column. Prevents update anomalies (changing "Snacks" to "Snack Foods" would require updating every row in the old design; now it's one UPDATE on FOOD_CATEGORY).

3.4.3 Table: DATA_TYPE

Column	Type	Constraints	Default	Description
type_id	INT	PK, IDENTITY(1,1)	auto	Auto-generated type identifier
type_name	VARCHAR(100)	NOT NULL, UNIQUE	—	Data source classification

Purpose: Normalized lookup for USDA data source types. Values include "SR Legacy" (Standard Reference), "Survey (FNDDS)", "Branded Foods", etc.

3NF Rationale: Same pattern as FOOD_CATEGORY — extract repeating classification strings to prevent update anomalies.

3.4.4 Table: Foods

Column	Type	Constraints	Default	D
fdc_id	INT	PK	—	U Fo Da Ce tra id ti- fie (n u- ra ke
brand_id	INT	FK → Brands(brand_id) ON DELETE SET NULL	NULL	Op br as so ci- a- tio
category_id	INT	FK → FOOD_CATEGORY(category_id)	—	Fo ca e- go re er- en
type_id	INT	FK → DATA_TYPE(type_id)	—	Da so ty re er- en
food_name	VARCHAR(500)	NOT NULL	—	De fo ite na

Purpose: Core entity. Stores every food item from the USDA database. fdc_id is a natural key (provided by USDA, not auto-generated). All descriptive attributes (category, type) are normalized into lookup tables via FK references.

Indexes:

- `idx_foods_brand_id` — speeds up brand JOINs
- `idx_foods_category_id` — speeds up category JOINs and filtering
- `idx_foods_type_id` — speeds up data type JOINs
- `idx_foods_food_name` — B-tree on food_name; helps prefix LIKE queries. Leading-wildcard queries (`LIKE '%Apple%'`) will still table-scan.

3.4.5 Table: Nutrition_Metrics

Column	Type	Constraints	Default	Des
nutrition_id	INT	PK, IDENTITY(1,1)	auto	Auto gene nu- tri- tion reco ID
fdc_id	INT	FK → Foods(fdc_id) ON DELETE CASCADE, UNIQUE	—	One- to- one link to par- ent food
calories	FLOAT	NULL	—	Ener con- tent in kcal
protein_g	FLOAT	NULL	—	Prot con- tent in gran
fat_g	FLOAT	NULL	—	Tota fat con- tent in gran
carbs_g	FLOAT	NULL	—	Carb con- tent in gran
sodium_mg	FLOAT	NULL	—	Sodi con- tent in mil- ligra

Purpose: Stores quantitative nutritional composition for each food. UNIQUE constraint on fdc_id enforces a strict 1:1 relationship — each food has exactly zero or one nutrition

record. NULL values are permitted for individual nutrients (some foods lack complete data).

Design Decision: Nutrition data is separated from the Foods table (rather than being columns on Foods) because: (a) it represents a distinct functional domain, (b) many foods have no nutrition data (sparse), and (c) it keeps the Foods table narrow for fast listing queries.

Data Quality: The `get_foods_with_missing_data` function identifies foods where any of these five columns are NULL, flagging them for data completion.

3.4.6 Table: HEALTH_SCORE

Column	Type	Constraints	De
score_id	INT	PK, IDENTITY(1,1)	au
fdc_id	INT	FK → Foods(fdc_id) ON DELETE CASCADE, UNIQUE	—
health_score	FLOAT	NULL	—
nutriscore_grade	VARCHAR(50)	NULL	—
nova_group	INT	NULL	—

Purpose: Stores food quality and classification metrics. Three distinct scoring dimen-

sions:

- **health_score:** A numeric composite score (0–100), higher is healthier. Computed from nutritional composition.
- **nutriscore_grade:** The European Nutri-Score system (A=best, E=worst). Used as training labels for the ML model.
- **nova_group:** The NOVA food processing classification (1=unprocessed, 4=ultra-processed). Populated from OpenFoodFacts enrichment data.

3NF Rationale: Split from the original `Health_and_Allergens` table. Health scoring metrics are functionally independent of allergen flags — a food's Nutri-Score doesn't determine whether it contains gluten.

3.4.7 Table: ALLERGEN_PROFILE

Column	Type	Constraints	Default	De
allergen_id	INT	PK, IDENTITY(1,1)	auto	Au ge al- ler ge pr file ID
fdc_id	INT	FK → Foods(fdc_id) ON DELETE CASCADE, UNIQUE	—	Or to- on lin to pa en foo
contains_gluten	BIT	NOT NULL	0	Gl pr en fla (0= 1=
contains_dairy	BIT	NOT NULL	0	Da pr en fla (0= 1=

Purpose: Binary allergen flags for dietary filtering. The `get_foods_by_diet` function filters on these columns. Foods without allergen data are treated as "no known allergens" (NULL-safe WHERE clauses).

3NF Rationale: Split from `Health_and_Allergens`. Allergen presence is an intrinsic

property of the food, not a derived score. A food's gluten content is independent of its Nutri-Score.

3.4.8 Table: ML_Predictions

Column	Type	Constraints	Default
prediction_id	INT	PK, IDENTITY(1,1)	auto
fdc_id	INT	FK → Foods(fdc_id) ON DELETE CASCADE	—
predicted_nutriscore	VARCHAR(50)	NULL	—
predicted_nova	INT	NULL	—
confidence_score	FLOAT	NULL	—
prediction_date	DATETIME	NOT NULL	GETDAT

Purpose: Stores the output of the ML inference pipeline. The neural network predicts Nutri-Score grades; NOVA group classification is not attempted (nutrition data alone cannot determine food processing level).

Important Note: `predicted_nova` is always NULL. NOVA classification measures food processing (Group 1 = unprocessed, Group 4 = ultra-processed), which cannot be determined from the five nutrition features (calories, protein, fat, carbs, sodium). A diet soda (0 calories) is NOVA 4; a raw avocado (high calories) is NOVA 1. Training on nutrition data would produce systematically invalid NOVA predictions.

Indexes: `idx_ml_predictions_fdc_id` — speeds up JOINs with Foods table.

3.5 Data Value Creation

The system creates value from raw data through several transformations:

1. **Normalization.** Raw CSV data with duplicated VARCHAR strings (categories, types) is transformed into a normalized 3NF schema with lookup tables. This eliminates redundancy, prevents update anomalies, and improves query performance through integer FK joins instead of string comparisons.

2. **Data Enrichment.** The USDA CSV provides nutrition data. The OpenFoodFacts CSV provides Nutri-Score grades, NOVA groups, and allergen flags. The `_load_health_csv` function matches products by name (shortest-match heuristic) and enriches the HEALTH_SCORE and ALLERGEN_PROFILE tables. The enrichment data is what makes the ML pipeline possible — it provides the labeled training data.

3. **ML-Predicted Nutri-Scores.** The neural network learns the mapping from nutrition features → Nutri-Score grade from labeled data. It then predicts grades for all foods (including those without existing grades), creating inferred health classifications at scale.

4. **Aggregated Analytics.** The `get_category_aggregation` function computes per-category nutritional averages, enabling comparative analysis (e.g., "Are Snacks higher in sodium than Vegetables?").

5. **Gap Detection.** The `get_foods_with_missing_data` function identifies data quality issues by surfacing records with NULL nutrition fields, enabling targeted data completion.

6. **Dietary Intelligence.** The allergen profile enables filtering by dietary restriction (gluten-free, dairy-free), creating immediate practical value for users with food allergies or intolerances.

4 Backend Documentation

4.1 File: `database.py` — Connection Management

Purpose: Provides the database connection layer. All database access flows through this module.

4.1.1 DB_CONFIG (Module Constant)

```
DB_CONFIG = {
    "server": "127.0.0.1", "port": "1433",
    "user": "SA", "password": "...",
    "database": "NutriQuery",
    "login_timeout": 10, "timeout": 30,
}
```

Listing 1: Database configuration dictionary

What it does: Central configuration dictionary for MSSQL connection parameters. Server, port, credentials, database name, and timeout values are defined once and consumed by `get_connection()`.

Why: Single source of truth for database connectivity. Changing the server or credentials requires editing one dictionary, not hunting through multiple files.

4.1.2 get_connection()

What it does: Creates a pymssql connection to the NutriQuery database with retry logic (3 attempts with exponential backoff: 0.5s, 1.0s delays). Raises the last `OperationalError` if all attempts fail.

Why: Transient network failures or database restarts should not crash the application. The retry loop with sleep between attempts prevents thundering-herd reconnection storms.

4.1.3 get_db()

What it does: FastAPI dependency generator. Yields a `(connection, cursor_as_dict)` tuple. The cursor is configured with `as_dict=True`, so all query results are dictionaries keyed by column name. The `finally` block guarantees cursor and connection cleanup.

Why: FastAPI's dependency injection system calls this function for every request that declares `db: DbDep`. Each request gets a fresh connection and cursor; they are automatically closed when the request completes. The `as_dict=True` cursor enables readable code like `row["food_name"]` instead of `row[1]`.

4.2 File: `schemas.py` — Pydantic Data Models

Purpose: Defines all request/response data models using Pydantic. FastAPI uses these for automatic request validation, response serialization, and OpenAPI documentation generation.

4.2.1 Brand Schemas

- `BrandBase` — `brand_name: str`, `brand_owner: Optional[str]`
- `BrandCreate(BrandBase)` — used for POST /brands/ request body
- `Brand(BrandBase)` — adds `brand_id: int`; used for API responses

4.2.2 Nutrition Schemas

- `NutritionBase` — `calories`, `protein_g`, `fat_g`, `carbs_g`, `sodium_mg` (all Optional[float])

- `Nutrition(NutritionBase)` — adds `nutrition_id: int, fdc_id: int`

4.2.3 Health Score Schemas

- `HealthScoreBase` — `health_score, nutriscore_grade, nova_group` (all Optional)
- `HealthScore(HealthScoreBase)` — adds `score_id: int, fdc_id: int`

4.2.4 Allergen Profile Schemas

- `AllergenProfileBase` — `contains_gluten: bool = False, contains_dairy: bool = False`
- `AllergenProfile(AllergenProfileBase)` — adds `allergen_id: int, fdc_id: int`

4.2.5 ML Prediction Schemas

- `MLPredictionBase` — `predicted_nutriscore, predicted_nova, confidence_score` (all Optional)
- `MLPrediction(MLPredictionBase)` — adds `prediction_id, fdc_id, prediction_date`

4.2.6 Food Schemas

- `FoodBase` — `food_name: str, food_category, data_type` (Optional). Note: `food_category` and `data_type` come from JOIN lookups, not columns on the Foods table.
- `Food(FoodBase)` — nested model with `brand, nutrition, health_score, allergen` sub-models
- `FoodSearchResult` — lightweight result: `fdc_id, food_name, food_category, brand_name`
- `FoodBrowseResult` — browse result with nutrition: adds `calories, protein_g, fat_g, carbs_g, sodium_mg`

4.2.7 AggregationResult Schema

`food_category: str, avg_calories: float, avg_protein: float, avg_fat: float, avg_carbs: float, item_count: int`

Why Pydantic: Pydantic provides automatic type coercion, validation, and JSON serialization. Invalid request bodies are rejected with clear error messages before reaching business logic. The response models generate accurate OpenAPI/Swagger documentation automatically.

4.3 File: `crud.py` — Data Access Layer

Purpose: Contains all SQL query logic as pure functions. Each function accepts `(conn, cursor)` and operates on the database. No FastAPI, HTTP, or Pydantic dependencies — pure database logic.

4.3.1 Shared Query Infrastructure

```
_FOOD_COLUMNS = (
    "f.fdc_id, f.food_name, f.brand_id, "
    "dt.type_name AS data_type, "
    "fc.category_name AS food_category, "
    "..."
)

_FOOD_FROM = (
    "FROM Foods f "
    "LEFT JOIN Brands b            ON f.brand_id    = b.brand_id "
    "LEFT JOIN FOOD_CATEGORY fc    ON f.category_id = fc.category_id "
    "LEFT JOIN DATA_TYPE dt        ON f.type_id     = dt.type_id "
    "LEFT JOIN Nutrition_Metrics n ON f.fdc_id      = n.fdc_id "
    "LEFT JOIN HEALTH_SCORE hs     ON f.fdc_id      = hs.fdc_id "
    "LEFT JOIN ALLERGEN_PROFILE ap ON f.fdc_id      = ap.fdc_id"
)
```

Listing 2: Shared query constants

What: Module-level string constants defining the complete 7-table food JOIN. The `_food_query(extra, top)` helper function builds a full SELECT statement from these constants plus optional WHERE/ORDER BY clauses and an optional TOP N limit.

Why: DRY (Don't Repeat Yourself). The 21-column SELECT list and 6 LEFT JOINs are defined once. Four functions (`get_food`, `get_foods_by_range`, `get_foods_by_diet`, `get_foods_with_missing_data`) share this query. Adding a column to the food profile requires editing only the constants.

4.3.2 _build_food_dict(row)

What it does: Transforms a flat JOIN result row (single dictionary with all columns) into a nested dictionary matching the `Food` Pydantic model structure. Conditionally nests `brand`, `nutrition`, `health_score`, and `allergen` sub-dictionaries based on whether the corresponding ID fields are non-NULL.

Why: The SQL JOIN returns a flat row (all columns at the same level). The API expects nested JSON (brand, nutrition, health, allergen as sub-objects). This function bridges the gap between the relational and hierarchical representations.

4.3.3 get_food(conn, cursor, fdc_id)

What: Retrieves a complete food profile by fdc_id. Executes the 7-table LEFT JOIN query with `WHERE f.fdc_id = %s`. Returns the nested food dict or None if not found.

Used by: `GET /foods/{fdc_id}` endpoint; BrowseSection row-click detail panel.

4.3.4 _update_row(conn, cursor, table, columns, fdc_id, data)

What: Generic UPDATE helper. Accepts a table name, a tuple of allowed columns, an fdc_id, and a data dict. Dynamically builds `SET col = %s` clauses only for columns present in the data dict. Executes the UPDATE, commits, and returns the updated row via SELECT.

Why: Eliminates code duplication. The three update functions (`update_nutrition`, `update_health_score`, `update_allergen`) are thin wrappers that differ only in table

name and column list. The generic helper reduces 63 lines of copy-paste to 15 lines + 3 thin wrappers.

4.3.5 update_nutrition(conn, cursor, fdc_id, data)

What: Updates `Nutrition_Metrics` for a food. Allowed columns: calories, protein_g, fat_g, carbs_g, sodium_mg.

4.3.6 update_health_score(conn, cursor, fdc_id, data)

What: Updates `HEALTH_SCORE` for a food. Allowed columns: health_score, nutriscore_grade, nova_group.

4.3.7 update_allergen(conn, cursor, fdc_id, data)

What: Updates `ALLERGEN_PROFILE` for a food. Allowed columns: contains_gluten, contains_dairy.

4.3.8 get_foods_by_range(conn, cursor, min_health_score, max_sodium, max_carbs, limit=100)

What: Range query filtering foods by simultaneous constraints on health score ($\geq$), sodium ($\leq$), and carbs ($\leq$). Uses the shared food query with WHERE clauses on `hs.health_score`, `n.sodium_mg`, and `n.carbs_g`. Results ordered by health score descending.

Used by: `GET /queries/range` endpoint; AnalyticsSection Range Query.

4.3.9 get_foods_by_diet(conn, cursor, no_gluten, no_dairy, limit=100)

What: Dietary filtering using allergen profiles. Constructs WHERE clauses with NULL-safe checks: `(ap.contains_gluten IS NULL OR ap.contains_gluten = 0)`. This ensures foods without allergen profiles are included (treated as "no known allergens").

Why NULL-safe: Many foods lack allergen profile data. An INNER JOIN on ALLERGEN_PROFILE would exclude these foods entirely from dietary filtering. The LEFT JOIN + NULL-safe WHERE ensures they pass the filter.

4.3.10 get_category_aggregation(conn, cursor, category)

What: Computes per-category nutritional averages using `AVG()` aggregation. Joins Foods $\rightarrow$ FOOD_CATEGORY $\rightarrow$ Nutrition_Metrics. Returns avg calories, protein, fat, carbs, and item count. Falls back to all-zero result if the category doesn't exist.

4.3.11 get_foods_with_missing_data(conn, cursor, limit=100)

What: Gap identification. Finds foods where any of the five nutrition columns is NULL. Uses the shared food query with OR-connected IS NULL checks.

4.3.12 create_brand(conn, cursor, data)

What: Inserts a new brand record. Uses `@@IDENTITY` to retrieve the auto-generated brand_id, then SELECTs the complete row to return.

4.3.13 get_brands(conn, cursor, skip, limit)

What: Paginated brand listing using `OFFSET/FETCH NEXT` (MSSQL's equivalent of LIMIT/OFFSET).

4.3.14 search_foods(conn, cursor, name, limit)

What: Substring search on food_name using `LIKE '%name%'`. Returns lightweight results (fdc_id, food_name, category, brand_name) without nutrition data. Leading and trailing wildcards enable infix matching.

4.3.15 get_foods_browse(conn, cursor, skip, limit, name)

What: Paginated food listing WITH nutrition data for the browse view. Supports optional name filtering (switches from `ORDER BY fdc_id` to `ORDER BY food_name` when filtering). Returns `FoodBrowseResult` schema (includes calories, protein, fat, carbs, sodium).

Why separate from search: The search endpoint returns lightweight results without nutrition (for fast autocomplete-style queries). The browse endpoint includes nutrition data (for the main grid where users see nutrition at a glance). Both use the same underlying pattern but serve different UX needs.

4.3.16 get_all_foods(conn, cursor, skip, limit)

What: Simple paginated listing without nutrition data. Returns `FoodSearchResult` schema.

4.3.17 get_categories(conn, cursor)

What: Lists all distinct category names from the FOOD_CATEGORY table, sorted alphabetically. Used to populate category dropdown menus.

4.3.18 get_predictions(conn, cursor, limit)

What: Fetches ML predictions joined with food names. Uses LEFT JOIN on Foods so orphaned predictions still appear (with NULL food_name).

4.4 File: `main.py` — API Endpoints

Purpose: FastAPI application definition. Maps HTTP routes to CRUD functions. Handles request validation, error responses, CORS, and background tasks.

4.4.1 Application Setup

- `app = FastAPI(title="NutriQuery API", version="1.1.0")`
- CORS middleware: allows origins `localhost:5173` and `127.0.0.1:5173` (Vite dev server)
- `DbDep = Annotated[tuple, Depends(get_db)]` — dependency injection type alias
- ML router included via `app.include_router(ml_service.ml_router)`

4.4.2 Endpoint Catalog

Endpoint	Method	Description & Implementation
`/`	GET	Health check. Returns welcome message.
`/foods/search`	GET	Food search by name substring. Params: `name` (min_length=1), `limit` (1–200). Calls `crud.search_foods()`.
`/foods/browse`	GET	Paginated food listing with nutrition. Params: `skip, limit, name` (optional). Calls `crud.get_foods_browse()`.
`/foods/`	GET	Paginated food listing (no nutrition). Params: `skip, limit`. Calls `crud.get_all_foods()`.
`/foods/\{fdc\_id\}`	GET	Single food detail. Returns 404 if not found. Calls `crud.get_food()`.
`/foods/\{fdc\_id\}/nutrition`	PUT	Update nutrition fields. Validates body has at least one field; returns 400 if empty, 404 if record missing.
`/foods/\{fdc\_id\}/health`	PUT	Update health score fields. Same validation pattern as nutrition.
`/foods/\{fdc\_id\}/allergen`	PUT	Update allergen flags. Always accepts the body (both fields default to False).
`/queries/range`	GET	Range query. Params: `min_health_score, max_sodium, max_carbs, limit` (1–500).
`/queries/dietary`	GET	Dietary filter. Params: `no_gluten, no_dairy, limit` (1–500).
`/queries/aggregation`	GET	Category aggregation. Param: `category` (auto-selects first available if omitted).
`/queries/gaps`	GET	Gap identification. Param: `limit` (1–500).
`/brands/`	POST	Create a new brand. Body: `{brand_name, brand_owner}`.
`/brands/`	GET	List brands. Params: `skip, limit`.
`/categories/`	GET	List all distinct food categories.
`/predictions/`	GET	List ML predictions. Param: `limit`.
`/import`	POST	Trigger CSV data import (runs in background). Returns immediately; check `/import/status`.
`/import/status`	GET	Check background import status. Returns `{running, elapsed_seconds, last_result}`.

`/ml/predict`	POST	Train model and run inference. Returns 400 if <10 labeled samples exist.
`/ml/predictions`	DELETE	Delete all prediction records.
`/ml/device`	GET	Report ML compute device (cpu/cuda/mps).

Table 1: Complete API Endpoint Reference

4.4.3 Import State Tracking

What: Module-level `_import_state` dictionary tracks background import progress. `_run_import()` sets timestamps and captures the result. The `/import/status` endpoint exposes this state.

Why: The import can run for 10+ minutes. Users need visibility into whether it's running, how long it's been running, and what happened when it finished. The per-process state note warns about multi-worker deployments.

Limitation: State is per-process. Under `uvicorn -workers N`, each worker has its own copy. Single-worker deployment recommended for accurate status.

4.5 File: `ml_service.py` — Machine Learning Pipeline

Purpose: PyTorch-based neural network for Nutri-Score classification. Trains on labeled data from the database, runs inference on all foods, and stores predictions.

4.5.1 Device Selection: _get_device()

What: Lazily detects and caches the best available compute device. Priority: CUDA → MPS (Apple Silicon) → CPU. Uses module-level caching so detection runs once.

Why: PyTorch device selection at import time can be unreliable in container/cloud environments where CUDA initializes asynchronously. Lazy evaluation ensures detection happens when the model is actually needed.

4.5.2 Model Architecture: NutriScorePredictor

What: A feed-forward neural network with architecture: `Linear(5, 32)` → `ReLU` → `Dropout(0.2)` → `Linear(32, 16)` → `ReLU` → `Linear(16, 5)`. Input: 5 nutrition features (calories, protein, fat, carbs, sodium). Output: 5-class logits mapped to Nutri-Score grades A–E.

Why this architecture: Small and fast — appropriate for the dataset size (tens to hundreds of labeled samples). The dropout layer provides basic regularization. The 32→16 bottleneck forces the model to learn compressed representations.

Training hyperparameters: Adam optimizer (lr=0.005), CrossEntropyLoss, StepLR scheduler (step_size = max(10, epochs/3), gamma=0.5). Epochs scale with data: `min(200, max(30, samples * 5))`.

4.5.3 _compute_normalization_stats(cursor)

What: Computes per-feature mean and standard deviation from ALL foods with complete nutrition data (not just labeled data). Builds a full tensor in memory. Returns `(mean, std)` tensors.

Why full dataset: Training only sees labeled data (a subset). If normalization statistics were computed from the labeled subset only, inference on the full dataset would use mismatched statistics. Computing from the full dataset ensures training and inference use the same normalization parameters.

4.5.4 _train_model(cursor)

What: Creates a fresh model instance, computes normalization stats, fetches labeled training data (foods with nutriscore_grade A–E AND complete nutrition), trains for `epochs` iterations with learning rate scheduling, and returns `(model, True)`. Returns `(None, False)` if fewer than 10 labeled samples exist.

Why fresh model each call: Avoids thread-safety issues. If multiple requests hit `/ml/predict` simultaneously, each gets its own model instance rather than sharing mutable state.

Early stopping: If loss becomes NaN (numerical instability), training breaks immediately.

4.5.5 run_inference_and_store(conn, cursor)

What: Orchestrates the full ML pipeline:

1. Call `_train_model()` — raises HTTPException(400) if training fails
2. Fetch all foods with nutrition data for inference
3. Count and delete old predictions
4. For each food: normalize features, run forward pass, get softmax confidence, extract predicted class
5. Insert prediction with NULL nova (NOVA not predicted)
6. Batch commit every 500 rows

4.5.6 delete_predictions(conn, cursor)

What: Counts and deletes all predictions. Returns the count in the response message.

4.5.7 ML Router

Three endpoints mounted at `/ml` prefix:

- `POST /ml/predict` — triggers training + inference
- `DELETE /ml/predictions` — clears predictions
- `GET /ml/device` — reports compute device

4.6 File: `data_import.py` — CSV Data Import

Purpose: Reads USDA and OpenFoodFacts CSV files from `data/raw/` and populates all 8 tables of the 3NF schema using raw SQL INSERT statements.

4.6.1 Value Helpers

- `_safe_float(val)` — coerces to float, returns None on failure. Rejects `inf` (via `abs(f) != float('inf')`) and `NaN` (via `f == f` — NaN is the only float where `x != x`).

- `_safe_int(val)` — coerces to int via float intermediate. Rejects `inf` and handles `OverflowError` from extreme values.

Why: CSV files contain dirty data — empty strings, non-numeric values, infinity representations. These helpers ensure only valid numbers reach the database, preventing INSERT failures and NaN-corrupted aggregations.

4.6.2 _resolve_or_create(cursor, cache, table, id_col, name_col, name_value, extra_cols=None)

What: Generic lookup-or-create pattern. Checks an in-memory cache first, then the database, and inserts if not found. Returns the ID. Used for Brands, FOOD_CATEGORY, and DATA_TYPE. The `extra_cols` parameter handles tables with additional columns beyond the name (e.g., Brands has `brand_owner`).

Why: Eliminates three copies of identical lookup-or-create logic. The in-memory cache avoids repeated database lookups for values already seen in the current import batch.

4.6.3 _load_usda_csv(conn, cursor, stats)

What: Reads `comprehensive_foods_usda.csv` row by row using `csv.DictReader`. For each row:

1. Resolves brand, category, and type via `_resolve_or_create` (with in-memory caching)
2. Checks if the food already exists (SELECT before INSERT for idempotency)
3. Inserts into Foods, Nutrition_Metrics, HEALTH_SCORE, ALLERGEN_PROFILE
4. Commits every 500 rows
5. Tracks stats: foods, brands, categories, data_types, nutrition, health, allergens

Why SELECT-before-INSERT: Makes the import idempotent — safe to re-run without creating duplicates. The trade-off is ∼240K SELECT statements for 40K rows. A future optimization would use `MERGE` or `INSERT WHERE NOT EXISTS`.

4.6.4 _load_health_csv(conn, cursor)

What: Reads `foods_health_scores_allergens.csv`. Matches products by `product_name` → `food_name` using LIKE prefix search with deterministic ordering (`ORDER BY LEN(food_name) ASC, food_name ASC`). Updates HEALTH_SCORE (nutriscore_grade, nova_group) and ALLERGEN_PROFILE (contains_gluten, contains_dairy). Returns counts of updated records.

Why shortest-match: "Apple" should match "Apple, raw" (length 10) before "Apple pie filling" (length 18). The deterministic ordering ensures the same product always matches the same food across re-imports.

4.6.5 import_all_data()

What: Orchestrator. Opens a connection, runs both CSV loaders, captures enrichment counts, commits, and returns a summary dict with all statistics. Wraps everything in try/except with rollback on failure and guaranteed cleanup in finally.

Why: Single entry point. Called by `POST /import` (background task) and `python data_import.py` (CLI).

5 Frontend Documentation

5.1 File: `api.js` — API Client

Purpose: Centralized HTTP client for all frontend-to-backend communication.

5.1.1 fetchAPI(endpoint, options)

What: Wraps the Fetch API. Prepends `http://localhost:8000` to the endpoint path. On HTTP errors, attempts to parse a JSON `detail` field; falls back to status code. Returns the parsed JSON or `{error: message}` on failure.

Why: Single point for error handling, base URL configuration, and request formatting. All components use this function instead of raw fetch — changing the API base URL requires one edit.

5.2 File: `App.jsx` — Application Shell

Purpose: Root React component. Renders the header, five section components, and footer.

Component tree:

```
App
 Header (NutriQuery logo + subtitle)
 BrowseSection   (food browser, search, detail, editing)
 AnalyticsSection (range query, dietary filter, aggregation, gaps)
 MLEngineSection  (ML training, predictions, device info)
 BrandsSection    (brand CRUD)
 DataImportSection (CSV import trigger)
 Footer
```

5.3 File: `BrowseSection.jsx` — Food Browser

Purpose: Primary food interaction component. Combines search, browse, detail view, and inline editing.

State variables:

- `searchName` — current search text
- `rowData` — grid data (flattened for AG Grid)
- `page` — current pagination page number
- `isBrowsing` — true when browsing all foods (shows pagination)
- `selectedFood` — the food currently shown in the detail panel
- `editing` — which section is being edited ('nutrition' | 'health' | 'allergen' | null)
- `editForm` — form field values during editing

5.3.1 loadPage(pageNum, searchTerm)

What: Fetches a page of food data from `/foods/browse`. If a search term is provided, appends it as the `name` parameter. Flattens the API response for AG Grid consumption.

Why unified endpoint: Both browse and search use the same `/foods/browse` endpoint. The `name` parameter distinguishes search from browse. This ensures search

results include nutrition data (unlike the old `/foods/search` endpoint which returned lightweight results).

5.3.2 onRowClicked(event)

What: AG Grid row click handler. Fetches the full food detail from `GET /foods/{fdc_id}` and displays it in the detail panel below the grid.

Why: The browse grid shows summary data (name, category, calories, etc.). Clicking a row reveals the full profile with nutrition breakdown, health scores, allergen flags, and brand metadata.

5.3.3 openEdit(section) / saveEdit()

What: Opens an inline edit form for nutrition, health, or allergens. Pre-populates with current values. On save, sends a PUT request to the appropriate endpoint and refreshes the detail view.

Why: Enables data correction (Requirement 3) directly from the UI. Users don't need to use curl or a separate admin panel to fix incorrect nutrition data.

5.3.4 AG Grid Configuration

Columns: fdc_id (90px), food_name (flex:2, word-wrapped), food_category (flex:1), brand_name (flex:1), calories (75px), protein_g (85px), fat_g (80px), carbs_g (85px), sodium_mg (85px).

Features: Auto-height rows for wrapped food names, value formatters for numeric precision, click-to-detail, overlay templates for loading/empty states.

5.4 File: `AnalyticsSection.jsx` — Nutrition Analytics

Purpose: Exposes the four analytical query endpoints (Requirements 4–7) through a structured form-based interface.

5.4.1 Range Query (Req 4)

Controls: Min health score slider (0–100), max sodium input, max carbs input, execute button. **Results grid:** Shows fdc_id, food_name, food_category, health_score (color-coded: green ≥ 60, red < 60), nutriscore_grade, calories, sodium_mg, carbs_g. **Why show values:** Users can verify that the returned foods actually meet their filter criteria. The health score is color-coded for instant visual assessment.

5.4.2 Dietary Filtering (Req 5)

Controls: Gluten-free checkbox, dairy-free checkbox, filter button. **Results grid:** Shows fdc_id, food_name, food_category, gluten status (Yes/No, color-coded), dairy status.

5.4.3 Category Aggregation (Req 6)

Controls: Category dropdown (populated from `GET /categories/`), aggregate button. **Results:** Five stat cards showing item count, avg calories, avg protein, avg fat, avg carbs.

5.4.4 Gap Identification (Req 7)

Controls: Identify Gaps button. **Results grid:** Shows nutrition columns with NULL values highlighted in red and labeled "MISSING."

5.5 File: `MLEngineSection.jsx` — ML Engine

Purpose: Interface for the PyTorch ML pipeline.

Features:

- Device badge showing current compute device (CPU/CUDA/MPS Accelerated)
- Train & Predict button (calls `POST /ml/predict`)
- Flush Predictions button (calls `DELETE /ml/predictions`)
- View Predictions button (calls `GET /predictions/`)
- Predictions grid: fdc_id, food_name, predicted Nutri-Score, confidence (formatted as percentage)

Note: The NOVA column was intentionally removed from the grid because `predicted_nova` is always NULL — NOVA classification requires food processing metadata, not nutritional composition.

5.6 File: `BrandsSection.jsx` — Brand Management

Purpose: CRUD interface for food brands (Requirement 8).

Features:

- Create form: brand_name (required), brand_owner (optional)
- Brands list: AG Grid with brand_name and brand_owner columns, paginated
- Refresh button to reload the brands list

5.7 File: `DataImportSection.jsx` — Data Import

Purpose: Trigger for the CSV data import pipeline.

Features:

- Run Full Import button (calls `POST /import`)
- Loading state with disabled button during import
- Status message display with import results

5.8 File: `index.css` — Design System

Purpose: Global stylesheet implementing a creamy white matte design system with CSS custom properties.

Design tokens: Colors (bg, panel, text-primary/secondary, border, primary, accent, success, warning, danger), typography (Inter for body/headings, JetBrains Mono for monospace), shadows (sm/md/lg), border radius, transitions.

Component classes: `matte-panel`, `sub-panel`, `detail-panel`, `detail-card`, `detail-grid`, `stat-grid`, `stat-card`, `form-grid`, `button-row`, `checkbox-row`, `two-col`, `grid-container`, `pagination-row`, `device-badge`, `status-message`, `error-message`.

Responsive: `@media (max-width: 768px)` breakpoint with column-to-stack layout changes, full-width buttons, and single-column grids.

Word-wrap fix: `.ag-theme-quartz .ag-cell { white-space: normal !important; word-break: break-word; }` ensures long food names don't overflow grid cells.

6 Testing Infrastructure

6.1 File: `conftest.py` — Test Fixtures

Purpose: Configures a dedicated `NutriQuery_Test` database for isolated integration testing.

6.1.1 test_db_setup (Session Scope)

What: Creates the test database, runs `init.sql` (parsed with regex GO-splitting), seeds known test data, and drops the database at session end. Seed data is committed (visible to all connections, including the API TestClient).

6.1.2 db_conn / db_cursor (Function Scope)

What: Provides a connection with explicit transaction isolation (`BEGIN TRAN` / `ROLLBACK`). Test modifications are automatically rolled back.

6.1.3 client (Function Scope)

What: FastAPI TestClient pointed at the test database (monkey-patches `DB_CONFIG["database"]`).

6.1.4 seed_test_data

What: Returns the IDs generated during session setup (brand_id, cat_id, cat2_id, type_id) so tests can reference them.

Test Data: 5 foods (1001–1005), 1 brand, 2 categories (Snacks, Dairy), 1 data type (SR Legacy). Food 1004 has all-NULL nutrition (gap detection target). Food 1005 has no ALLERGEN_PROFILE row (LEFT JOIN test target). Health scores A–D provide labeled training data.

6.2 Test Files

- `test_api.py` (9 tests) — All API endpoint structure validation
- `test_crud.py` (6 tests) — Direct CRUD function testing
- `test_fixes.py` (9 tests) — Regression tests for each bug fix from the forensic audit
- `test_integration.py` (10 tests) — End-to-end flows: food retrieval, updates, dietary filtering, ML pipeline
- `test_ml.py` (10 tests) — Device detection, model creation, forward pass, untrained rejection, full ML pipeline
- `test_import.py` (4 tests) — Value coercion, CSV parsing, module structure

Total: 48 tests, all passing. No mocks — every test connects to a real MSSQL instance.

7 API Reference Summary

Endpoint	Method	Key Parameters
/	GET	—
/foods/search	GET	name (str, min_len=1), limit (1–200)
/foods/browse	GET	skip (int), limit (1–200), name (str, optional)
/foods/	GET	skip (int), limit (1–200)
/foods/\{id\}	GET	fdc_id (int, path)
/foods/\{id\}/nutrition	PUT	NutritionBase body
/foods/\{id\}/health	PUT	HealthScoreBase body
/foods/\{id\}/allergen	PUT	AllergenProfileBase body
/queries/range	GET	min_health_score, max_sodium, max_carbs, limit
/queries/dietary	GET	no_gluten, no_dairy, limit
/queries/aggregation	GET	category (default: auto-select)
/queries/gaps	GET	limit (1–500)
/brands/	POST	BrandCreate body
/brands/	GET	skip, limit (1–500)
/categories/	GET	—
/predictions/	GET	limit (1–500)
/import	POST	— (runs in background)
/import/status	GET	—
/ml/predict	POST	— (returns 400 if <10 labeled)
/ml/predictions	DELETE	—
/ml/device	GET	—

Table 2: API Quick Reference

8 Data Flow Diagrams

8.1 Import Flow

```
CSV files (data/raw/)


data_import.py
     _safe_float / _safe_int  (value sanitization)
     _resolve_or_create       (brand/category/type lookup)
     _load_usda_csv           (40K foods → 6 tables)
     _load_health_csv         (enrichment → HEALTH_SCORE, ALLERGEN_PROFILE)
```

```
MSSQL NutriQuery (8 tables, 3NF)
```

8.2 Query Flow

```
Browser (React)
      fetchAPI()

FastAPI (main.py)
      Depends(get_db)

crud.py
      _food_query() → pymssql

MSSQL (7-table JOIN)
      flat row dict

_build_food_dict()
      nested dict

Pydantic serialization
      JSON

AG Grid rendering
```

8.3 ML Flow

```
POST /ml/predict


_train_model()
     _compute_normalization_stats()  (full dataset mean/std)
     Fetch labeled data (nutriscore_grade A-E)
     Train NutriScorePredictor (epochs = f(samples))
     Return (model, True)


run_inference_and_store()
     Fetch all foods with nutrition
     DELETE old predictions
     For each food: normalize → forward → softmax → INSERT
     Commit
```

8.4 Edit Flow

```
User clicks Edit (BrowseSection)


openEdit('nutrition')
```

```
     Pre-fill form with current values


User modifies values, clicks Save


saveEdit()
     PUT /foods/{id}/nutrition
     API validates body, calls crud.update_nutrition()
     _update_row() → UPDATE Nutrition_Metrics → COMMIT → SELECT
     Response → refresh detail panel
```

9 Index of Functions

Function	File	Purpose
get_connection()	database.py	Create pymssql connection with retry
get_db()	database.py	FastAPI dependency: yield (conn, cursor)
_safe_float(val)	data_import.py	Coerce to float, reject inf/nan
_safe_int(val)	data_import.py	Coerce to int, reject inf
_resolve_or_create(...)	data_import.py	Generic lookup-or-create with cache
_load_usda_csv(conn, cursor, stats)	data_import.py	Import USDA CSV into 6 tables
_load_health_csv(conn, cursor)	data_import.py	Enrich health/allergen from CSV
import_all_data()	data_import.py	Orchestrator: run both imports
_food_query(extra, top)	crud.py	Build food SELECT from shared constants
_build_food_dict(row)	crud.py	Transform flat JOIN row to nested dict
_update_row(conn, cursor, ...)	crud.py	Generic UPDATE helper
get_food(conn, cursor, fdc_id)	crud.py	Full food profile via 7-table JOIN
update_nutrition(...)	crud.py	Update Nutrition_Metrics
update_health_score(...)	crud.py	Update HEALTH_SCORE
update_allergen(...)	crud.py	Update ALLERGEN_PROFILE
get_foods_by_range(...)	crud.py	Filter by health/sodium/carbs
get_foods_by_diet(...)	crud.py	Filter by allergen restrictions
get_category_aggregation(...)	crud.py	Per-category nutritional averages
get_foods_with_missing_data(...)	crud.py	Find incomplete nutrition records
create_brand(...)	crud.py	Insert brand with @@IDENTITY
get_brands(...)	crud.py	Paginated brand listing
search_foods(...)	crud.py	LIKE search on food_name
get_foods_browse(...)	crud.py	Paginated listing with nutrition
get_all_foods(...)	crud.py	Paginated listing (no nutrition)

get_categories(...)	crud.py	List distinct category names
get_predictions(...)	crud.py	List ML predictions with food names
_get_device()	ml_service.py	Lazy CUDA/MPS/CPU detection
_compute_normalization_stats(cursor)	ml_service.py	Full-dataset mean/std
_train_model(cursor)	ml_service.py	Train fresh NutriScorePredictor
run_inference_and_store(conn, cursor)	ml_service.py	Full ML pipeline
delete_predictions(conn, cursor)	ml_service.py	Clear all predictions
loadPage(pageNum, searchTerm)	BrowseSection.jsx	Paginated food fetch
onRowClicked(event)	BrowseSection.jsx	Row click → load detail
openEdit(section)	BrowseSection.jsx	Open inline edit form
saveEdit()	BrowseSection.jsx	PUT update + refresh
runRangeQuery()	AnalyticsSection.jsx	Execute range filter
runDietaryFilter()	AnalyticsSection.jsx	Execute dietary filter
runAggregation()	AnalyticsSection.jsx	Execute category aggregation
loadGaps()	AnalyticsSection.jsx	Load gap data
runInference()	MLEngineSection.jsx	Trigger ML training
clearPredictions()	MLEngineSection.jsx	Delete all predictions
createBrand()	BrandsSection.jsx	POST new brand
importData()	DataImportSection.jsx	Trigger CSV import
fetchAPI(endpoint, options)	api.js	Centralized HTTP client

Table 3: Complete Function Index